Ebook hub - end-to-end projects

1. ***Create guideline/prd***

Go to ChatGPT/Grok/Claude to generate guidelines for Cursor/Github Copilot:

The prompt:

You are tasked to give me the PRD or guideline for a project in markdown. The PRD should have as smallest tasks as possible because this PRD will be used in Cursor AI to develop a full-stack website for an Ebook store using the MERN stack.
The developing website will allow users to browse, search, and purchase books, while sellers can upload PDF files of their ebooks. This project is an MVP (Minimum Viable Product), so advanced features like payment processing, Nginx, or third-party services like cloud storage are not required for now. Instead, we will hard-code the payment logic.
The goal is to create a clear and actionable set of guidelines or a Product Requirements Document (PRD) to serve as a reference for Cursor AI to generate the code; therefore, the task should be as smallest as possible to avoid failures. The development machine will only have Docker installed, so the guidelines must emphasize that Docker should be used for running or debugging the application.
Key Requirements:

1. **Technology Stack:**
    - Full-stack web application based on the MERN stack (MongoDB, Express, React, Node.js) stack.
    - The website will have basic functionalities:
        - User features: Browse, search, and buy books.
        - Seller features: Upload PDFs of ebooks.
    - No advanced features such as:
        - Payment gateways (we will hard-code payment logic).
        - Nginx.

    - Third-party services (like cloud storage).

2. **Development Environment:**
    - The development machine will only have **Docker** installed.
    - All code should be compatible with Docker for running and debugging the application.
3. **Steps & Phases Breakdown:**
    - The development process should be divided into small tasks to ensure Cursor AI generates code step-by-step.
    - This step-by-step approach will help to avoid errors and facilitate debugging.

Guidelines for Cursor AI:

1. **Use Docker for Development and Debugging:**
    - Ensure that all steps and instructions are focused on using Docker.
    - All dependencies and services (like MongoDB, Node.js) should be configured in Docker containers.
    - Provide detailed steps for setting up Dockerfiles, docker-compose files, and any necessary configurations.
2. **MVP Functionality:**
    - Focus on building the basic functionalities first, without integrating advanced features.
    - Prioritize user authentication (login, register), book browsing, searching, and purchase capabilities.
    - For the seller's feature, allow uploading PDFs and storing metadata in MongoDB.
3. **Phase Breakdown:**
   Phase 1: Initial Setup
    - Set up the Docker environment.
    - Initialize the MERN stack project with necessary configurations.
    - Set up a basic MongoDB container.

   Phase 2: User Authentication

- Implement user registration and login functionality using JWT or sessions.
- Create the backend APIs and frontend UI for authentication.

Phase 3: Book Management

- Implement CRUD operations for books (admin/seller uploads).
- Allow users to browse and search for books.

Phase 4: Basic Purchase Flow

- Implement a hard-coded payment logic (no real payment gateway needed).
- Allow users to "purchase" a book, simulate the payment flow, and download the PDF.

Phase 5: Debugging and Testing

- Ensure all functionality works with Docker containers.
- Debug the application step-by-step, ensuring all features are correctly implemented.

Final Note: Please ensure that all code generated is structured and optimized for Docker. Since the development machine only has Docker installed, there should be no need for any other software or services outside of Docker containers.

---

2. ***We will got the output as below:***

Ebook Store MVP - Product Requirements Document (PRD)

Project Overview

Project Name: Ebook Store MVP

Technology Stack: MERN (MongoDB, Express.js, React.js, Node.js)

Development Environment: Docker-only

Target: Minimum Viable Product for ebook marketplace

Core Features

- **Users:** Browse, search, and purchase ebooks
- **Sellers:** Upload PDF ebooks with metadata
- **Hard-coded payment logic** (no real payment gateway)
- **Local file storage** (no cloud services)

Phase 1: Docker Environment Setup

Task 1.1: Create Project Structure

```
ebook-store/
├── docker-compose.yml
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
└── frontend/
    ├── Dockerfile
    ├── package.json
    └── src/
```

Task 1.2: Configure docker-compose.yml

- MongoDB service (port 27017)
- Backend service (port 5000)
- Frontend service (port 3000)
- Volume mapping for file uploads
- Network configuration between services

Task 1.3: Backend Dockerfile

- Use Node.js 18 alpine image
- Set working directory to /app
- Copy package.json and install dependencies
- Copy source code

- Expose port 5000
- Start with `npm start`

Task 1.4: Frontend Dockerfile

- Use Node.js 18 alpine image
- Set working directory to /app
- Copy package.json and install dependencies
- Copy source code
- Expose port 3000
- Start with `npm start`

Task 1.5: Test Docker Setup

- Run `docker compose up --build`
- Verify all containers start successfully
- Check MongoDB connection from backend

Phase 2: Backend Foundation

Task 2.1: Initialize Backend Project

- Create package.json with dependencies:
  - express
  - mongoose
  - cors
  - dotenv
  - multer (for file uploads)
  - bcryptjs
  - jsonwebtoken

Task 2.2: Basic Express Server Setup

- Create `src/app.js` with Express configuration
- Add CORS middleware

- Add JSON parsing middleware
- Create basic health check endpoint `/api/health`
- Add error handling middleware

Task 2.3: MongoDB Connection

- Create `src/config/database.js`
- Configure Mongoose connection to MongoDB container
- Add connection error handling
- Test connection on server start

Task 2.4: Environment Variables

- Create `.env` file with:
  - MONGODB_URI
  - JWT_SECRET
  - PORT
- Load environment variables in app.js

Phase 3: Data Models

Task 3.1: User Model

```
// src/models/User.js
{
  username: String (required, unique)
  email: String (required, unique)
  password: String (required, hashed)
  role: String (enum: ['user', 'seller'], default: 'user')
  createdAt: Date (default: Date.now)
}
```

Task 3.2: Book Model

```
// src/models/Book.js
{
```

```
  title: String (required)
  author: String (required)
  description: String
  category: String
  price: Number (required)
  pdfFile: String (file path)
  coverImage: String (file path, optional)
  sellerId: ObjectId (ref: User)
  createdAt: Date (default: Date.now)
}
```

Task 3.3: Purchase Model

```
// src/models/Purchase.js
{
  userId: ObjectId (ref: User)
  bookId: ObjectId (ref: Book)
  amount: Number
  status: String (enum: ['completed'], default: 'completed')
  purchaseDate: Date (default: Date.now)
}
```

Phase 4: Authentication System

Task 4.1: Auth Middleware

- Create `src/middleware/auth.js`
- JWT token verification
- User role checking middleware
- Request user attachment

Task 4.2: Auth Routes - Registration

- Create `src/routes/auth.js`
- POST `/api/auth/register` endpoint
- Validate input data

- Hash password with bcrypt
- Create user in database
- Return success message

Task 4.3: Auth Routes - Login

- POST `/api/auth/login` endpoint
- Validate credentials
- Compare hashed password
- Generate JWT token
- Return token and user info

Task 4.4: Auth Routes - Profile

- GET `/api/auth/profile` endpoint
- Require authentication middleware
- Return current user information

Phase 5: File Upload System

Task 5.1: Upload Configuration

- Create `src/config/upload.js`
- Configure multer for PDF uploads
- Set file size limits (10MB max)
- Define upload directory (`/app/uploads`)
- Add file type validation (PDF only)

Task 5.2: Upload Middleware

- Create file filter for PDF files
- Add filename sanitization
- Create unique filename generation
- Add error handling for upload failures

Task 5.3: Static File Serving

- Configure Express to serve static files
- Map `/uploads` route to uploads directory
- Add proper headers for file downloads

Phase 6: Book Management API

Task 6.1: Book Routes - Upload

- Create `src/routes/books.js`
- POST `/api/books` endpoint
- Require seller authentication
- Handle PDF file upload
- Save book metadata to database
- Return created book information

Task 6.2: Book Routes - Browse

- GET `/api/books` endpoint
- Add pagination (limit, skip)
- Add category filtering
- Add search by title/author
- Return book list with metadata

Task 6.3: Book Routes - Details

- GET `/api/books/:id` endpoint
- Return single book details
- Include seller information
- Exclude file path for security

Task 6.4: Book Routes - Download

- GET `/api/books/:id/download` endpoint
- Require user authentication

- Verify purchase (check Purchase model)
- Stream PDF file to user
- Add download logging

Phase 7: Purchase System

Task 7.1: Purchase Routes - Buy Book

- Create `src/routes/purchases.js`
- POST `/api/purchases` endpoint
- Require user authentication
- Validate book exists and price
- Hard-code payment success
- Create purchase record
- Return purchase confirmation

Task 7.2: Purchase Routes - User Purchases

- GET `/api/purchases/my` endpoint
- Require user authentication
- Return user's purchase history
- Include book details
- Add pagination

Task 7.3: Purchase Validation

- Check if user already owns book
- Prevent duplicate purchases
- Add purchase verification for downloads

Phase 8: Frontend Foundation

Task 8.1: Initialize React App

- Create package.json with dependencies:

- react
- react-dom
- react-router-dom
- axios
- bootstrap (for quick styling)

Task 8.2: App Structure

```
src/
├── components/
├── pages/
├── services/
├── context/
├── utils/
└── App.js
```

Task 8.3: API Service

- Create `src/services/api.js`
- Configure axios base URL
- Add token interceptor
- Add error handling
- Export API methods

Task 8.4: Auth Context

- Create `src/context/AuthContext.js`
- Manage user authentication state
- Provide login/logout functions
- Store JWT token in localStorage

Phase 9: Frontend Authentication

Task 9.1: Login Component

- Create `src/components/Login.js`
- Form with username/password fields
- Handle form submission
- Call login API
- Redirect on success
- Display error messages

Task 9.2: Register Component

- Create `src/components/Register.js`
- Form with username, email, password, role
- Handle form validation
- Call register API
- Redirect to login on success

Task 9.3: Protected Route Component

- Create `src/components/ProtectedRoute.js`
- Check authentication status
- Redirect to login if not authenticated
- Render component if authenticated

Phase 10: Frontend Book Display

Task 10.1: Book List Component

- Create `src/components/BookList.js`
- Display books in grid layout
- Show book cover, title, author, price
- Add "Buy" button for each book
- Handle pagination

Task 10.2: Book Detail Component

- Create `src/components/BookDetail.js`

- Display full book information
- Show purchase button
- Handle book purchase
- Show download link if owned

Task 10.3: Search Component

- Create `src/components/Search.js`
- Search input field
- Category filter dropdown
- Call search API on input change
- Update book list with results

Phase 11: Frontend Seller Features

Task 11.1: Upload Book Component

- Create `src/components/UploadBook.js`
- Form for book metadata input
- File input for PDF upload
- Progress indicator during upload
- Success/error message display

Task 11.2: Seller Dashboard

- Create `src/components/SellerDashboard.js`
- List seller's uploaded books
- Add basic sales statistics
- Link to upload new book

Phase 12: Frontend Routing

Task 12.1: Main App Component

- Create `src/App.js`

- Configure React Router
- Define all routes:
  - `/` - Home/Book List
  - `/login` - Login
  - `/register` - Register
  - `/book/:id` - Book Details
  - `/upload` - Upload Book (seller only)
  - `/dashboard` - User Dashboard
  - `/my-books` - Purchased Books

Task 12.2: Navigation Component

- Create `src/components/Navigation.js`
- Navigation bar with links
- Show different options for authenticated users
- Display user info and logout button

Phase 13: Purchase Flow Frontend

Task 13.1: Purchase Component

- Handle buy button click
- Show confirmation dialog
- Call purchase API
- Display success/error messages
- Update UI after purchase

Task 13.2: My Books Component

- Create `src/components/MyBooks.js`
- Display user's purchased books
- Add download buttons
- Handle PDF downloads

Phase 14: Error Handling & Validation

Task 14.1: Backend Error Handling

- Create custom error classes
- Add try-catch blocks to all routes
- Return consistent error responses
- Log errors for debugging

Task 14.2: Frontend Error Handling

- Create error boundary component
- Handle API error responses
- Display user-friendly error messages
- Add loading states

Task 14.3: Input Validation

- Add frontend form validation
- Validate file types and sizes
- Add backend validation middleware
- Sanitize user inputs

Phase 15: Testing & Debugging

Task 15.1: API Testing

- Test all endpoints with Postman/curl
- Verify authentication flows
- Test file upload functionality
- Check error responses

Task 15.2: Frontend Testing

- Test user registration/login
- Test book browsing and search

- Test purchase flow
- Test file uploads (seller)

Task 15.3: Docker Integration Testing

- Test with `docker-compose up`
- Verify container communication
- Test file persistence across restarts
- Check MongoDB data persistence

Phase 16: Final Polish

Task 16.1: Basic Styling

- Add Bootstrap classes for responsive design
- Style forms and buttons
- Create simple, clean layout
- Add loading spinners

Task 16.2: Documentation

- Create README.md with setup instructions
- Document API endpoints
- Add Docker commands reference
- Include troubleshooting guide

Task 16.3: Environment Cleanup

- Remove console.log statements
- Add proper environment variables
- Optimize Docker images
- Add health checks

Docker Commands Reference

```
# Build and start all services
docker compose up --build

# Start in background
docker compose up -d

# View logs
docker compose logs -f [service_name]

# Stop services
docker compose down

# Remove volumes (reset data)
docker compose down -v

# Rebuild specific service
docker compose build [service_name]
```

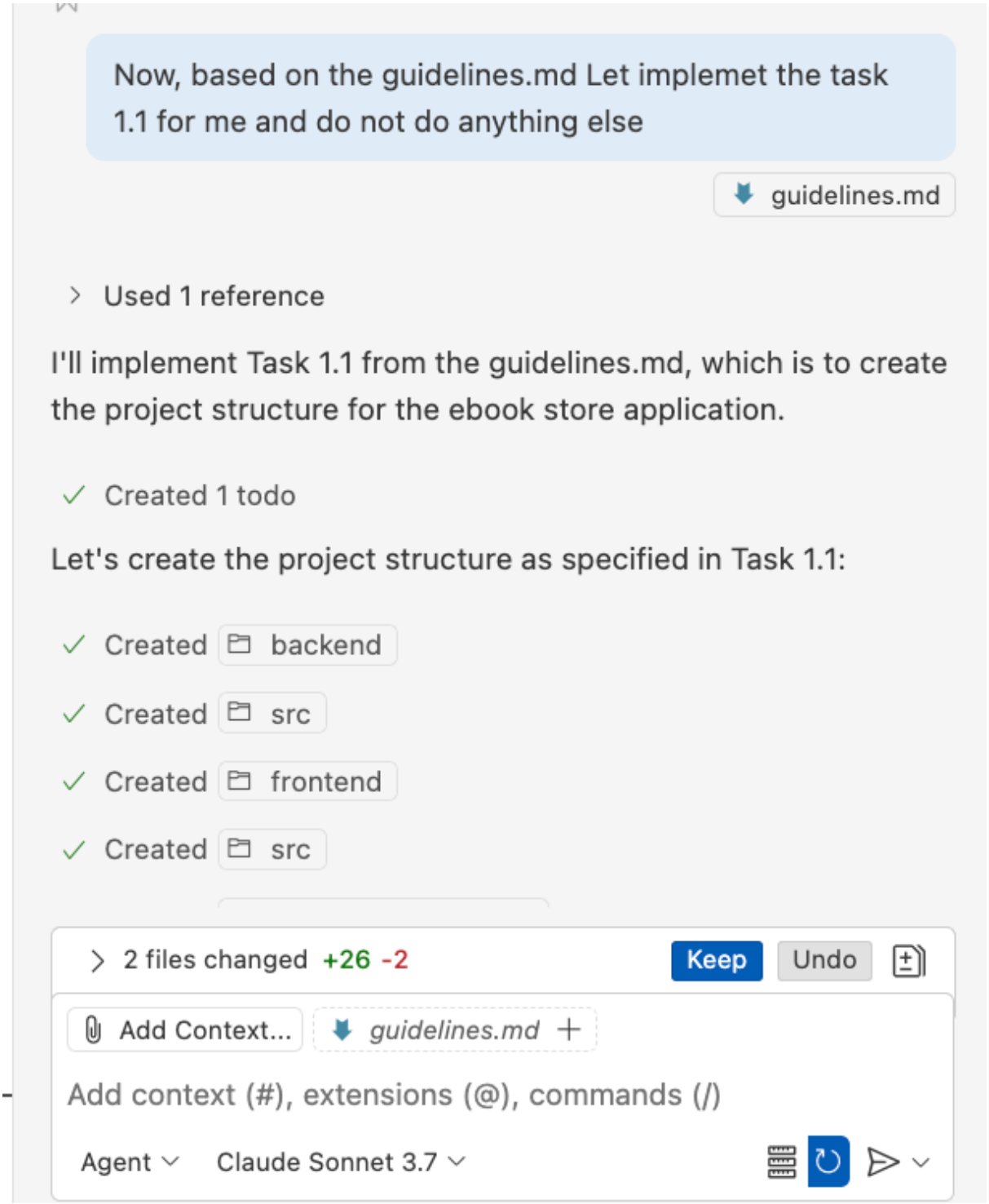
Success Criteria

- Users can register and login
- Sellers can upload PDF ebooks
- Users can browse and search books
- Users can "purchase" books with hard-coded payment
- Users can download purchased books
- All functionality works within Docker containers
- No external dependencies required

---

3. ***Vibe coding***

- We will review the output and then copy the contents into VS Code. Let create a file called “guidelines.md” then paste the contents into.
- In the Chat Tab of Cursor/Github Copilot → Choose Agent/Claude Sonnet 3.7
- Then we ask the agent to implement step by step. For instance:



The agent will create project structure for us.

- Next step is to implement task 1.2. The agent will create docker-compose.yml

And so on.

Until implementing task 1.5 we will check the docker container.

Using command to run:

```
docker compose up --build
```

Then we use command such as

```
docker ps -a
```

To list all the running container. If there is any error → just copy the error to the chat and ask the agent to fix.

- Then, we ask the agent to implement next Phase/tasks